

Cheat Sheet: Building Unsupervised Learning Models

1 Unsupervised learning models

Model Name	Brief Description	Code Syntax
UMAP	UMAP (Uniform Manifold Approximation and Projection) is used for dimensionality reduction. Pros: High performance, preserves global structure. Cons: Sensitive to parameters. Applications: Data visualization, feature extraction. Key hyperparameters: <code>n_neighbors</code> : Controls the local neighborhood size (default = 15). <code>min_dist</code> : Controls the minimum distance between points in the embedded space (default = 0.1). <code>n_components</code> : The dimensionality of the embedding (default = 2).	<code>from umap.umap_ import UMAP</code> <code>umap = UMAP(n_neighbors=15, min_dist=0.1, n_components=2)</code>
t-SNE	t-SNE (t-Distributed Stochastic Neighbor Embedding) is a nonlinear dimensionality reduction technique. Pros: Good for visualizing high-dimensional data. Cons: Computationally expensive, prone to overfitting. Applications: Data visualization, anomaly detection. Key hyperparameters: <code>n_components</code> : The number of dimensions for the output (default = 2). <code>perplexity</code> : Balances attention between local and global aspects of the data (default = 30). <code>learning_rate</code> : Controls the step size during optimization (default = 200).	<code>from sklearn.manifold import TSNE</code> <code>tsne = TSNE(n_components=2, perplexity=30, learning_rate=200)</code>

Model Name	Brief Description	Code Syntax
PCA	PCA (principal component analysis) is used for linear dimensionality reduction. Pros: Easy to interpret, reduces noise. Cons: Linear, may lose information in nonlinear data. Applications: Feature extraction, compression. Key hyperparameters: n_components: Number of principal components to retain (default = 2). whiten: Whether to scale the components (default = False). svd_solver: The algorithm to compute the components (default = 'auto').	1. <code>from sklearn.decomposition import PCA</code> 2. <code>pca = PCA(n_components=2)</code>
DBSCAN	DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm. Pros: Identifies outliers, does not require the number of clusters. Cons: Difficult with varying density clusters. Applications: Anomaly detection, spatial data clustering. Key hyperparameters: eps: The maximum distance between two points to be considered neighbors (default = 0.5). min_samples: Minimum number of samples in a neighborhood to form a cluster (default = 5).	1. <code>from sklearn.cluster import DBSCAN</code> 2. <code>dbscan = DBSCAN(eps=0.5, min_samples=5)</code>
HDBSCAN	HDBSCAN (Hierarchical DBSCAN) improves on DBSCAN by handling varying density clusters. Pros: Better handling of varying densities. Cons: Can be slower than DBSCAN. Applications: Large datasets, complex clustering problems. Key	1. <code>import hdbscan</code> 2. <code>clusterer = hdbscan.HDBSCAN(min_cluster_size=5)</code>

Model Name	Brief Description	Code Syntax
	hyperparameters:min_cluster_size: The minimum size of clusters (default = 5).min_samples: Minimum number of samples to form a cluster (default = 10).	
K-Means clustering	K-Means is a centroid-based clustering algorithm that groups data into k clusters.Pros: Efficient, simple to implement.Cons: Sensitive to initial cluster centroids.Applications: Customer segmentation, pattern recognition.Key hyperparameters:n_clusters: Number of clusters (default = 8).init: Method for initializing the centroids ('k-means++' or 'random', default = 'k-means++').n_init: Number of times the algorithm will run with different centroid seeds (default = 10).	1. <code>from sklearn.cluster import KMeans</code> 2. <code>kmeans = KMeans(n_clusters=3)</code>

2 Associated functions used

Method	Brief Description	Code Syntax
make_blobs	Generates isotropic Gaussian blobs for clustering.	1. <code>from sklearn.datasets import make_blobs</code> 2. <code>X, y = make_blobs(n_samples=100, centers=2, random_state=42)</code>
multivariate_normal	Generates samples from a multivariate normal distribution.	1. <code>from numpy.random import multivariate_normal</code> 2. <code>samples = multivariate_normal(mean=[0, 0], cov=[[1, 0], [0, 1]], size=100)</code>
plotly.express.scatter_3d	Creates a 3D scatter plot using Plotly Express.	1. <code>import plotly.express as px</code>

Method	Brief Description	Code Syntax
		2. <code>fig = px.scatter_3d(df, x='x', y='y', z='z')</code> 3. <code>fig.show()</code>
<code>geopandas.GeoDataFrame</code>	Creates a GeoDataFrame from a Pandas DataFrame.	1. <code>import geopandas as gpd</code> 2. <code>gdf = gpd.GeoDataFrame(df, geometry='geometry')</code>
<code>geopandas.to_crs</code>	Transforms the coordinate reference system of a GeoDataFrame.	1. <code>gdf = gdf.to_crs(epsg=3857)</code>
<code>contextily.add_basemap</code>	Adds a basemap to a GeoDataFrame plot for context.	1. <code>import contextily as ctx</code> 2. <code>ax = gdf.plot(figsize=(10, 10))</code> 3. <code>ctx.add_basemap(ax)</code>
<code>pca.explained_variance_ratio_</code>	Returns the proportion of variance explained by each principal component.	1. <code>from sklearn.decomposition import PCA</code> 2. <code>pca = PCA(n_components=2)</code> 3. <code>pca.fit(X)</code> 4. <code>variance_ratio = pca.explained_variance_ratio_</code>